

Lezione 1 - Applicazioni cloud a microservizi

Introduzione

Prima di introdurre i microservizi, vale la pena riprendere il filo di quanto fatto finora usando come riferimento l'esercizio della Rubrica Telefonica.

L'esercizio è stato impostato come una **applicazione web a servizi** — e il flusso di una richiesta segue esattamente l'architettura che abbiamo studiato:

1. **Postman(in questo caso il Client):**

- invia una **richiesta HTTP** con un payload **JSON** alla Controller

2. La **Controller:**

- **riceve il JSON,**
- **Spring lo deserializza in un oggetto DTO e la Controller diventa a sua volta client verso la Service**

3. La **Service:**

- **applica la logica di business e delega al DAO le operazioni sui dati**

4. Il **DAO:**

- **che fa da ponte con il "database" (la HashMap) — recupera i dati e li restituisce alla Service come Entity.**

5. La **Service:**

- **converte le Entity in DTO tramite il Mapper e li restituisce alla Controller**

6. La **Controller:**

- **restituisce il DTO a Spring che lo serializza in JSON e lo rimanda a Postman**

Un punto cruciale di questa architettura è l'**interoperabilità**:

- client e server si scambiano JSON;
 - **un formato generico non legato a nessuna tecnologia specifica.**
 - **Non importa se il client è scritto in JavaScript, Swift o Python — parla sempre JSON.**
 - **Ed è Spring che si mette in mezzo, occupandosi automaticamente della serializzazione e deserializzazione tra oggetti Java e JSON.**

Riepilogo dell'Architettura

Prima di passare ai microservizi, facciamo un riepilogo dei componenti dell'architettura che abbiamo costruito:

- **Entity:**
 - rappresenta l'entità sul database:
 - ogni classe corrisponde a una tabella, ogni oggetto a una riga
- **DTO:**
 - rappresenta il dato da trasferire, sia in input (dal client alla Controller) che in output (dalla Controller al client)
- **DAO / Repository:**
 - gestisce la persistenza:
 - contiene le operazioni CRUD verso il database
- **Service:**
 - contiene la logica di business
- **Controller:**
 - è l'interfaccia con il client:
 - riceve le richieste HTTP e restituisce le risposte JSON

L'Evoluzione con Hibernate

In un progetto reale con **Hibernate** l'architettura si semplifica notevolmente. Hibernate è un **ORM** (Object Relational Mapping):

- lo strumento moderno per gestire la persistenza in modo efficiente. Come abbiamo già accennato, con Hibernate non si scrive più il DAO manualmente — Hibernate lo genera automaticamente a partire dalle Entity.

Quindi in un progetto con Hibernate:

- **Entity:**
 - rimane, anzi diventa ancora più importante perché Hibernate la usa per generare automaticamente le query SQL
- **DTO:**
 - rimane per trasportare i dati verso il client
- **DAO:**
 - si scrive solo l'**interfaccia**:
 - Hibernate si occupa di generare automaticamente la classe concreta con tutte le operazioni CRUD

- **Service:**
 - rimane invariata:
 - **contiene la logica di business**
- **Controller:**
 - rimane invariata:
 - **gestisce le richieste HTTP**

☰ In sintesi:

la parte che viene sostituita da Hibernate è la **classe concreta del DAO**:

- **quella che nell'esercizio della rubrica avevamo scritto a mano con la HashMap.**

Con Hibernate basta definire l'interfaccia e il resto viene generato automaticamente.

I Microservizi

Dal Monolite ai Microservizi

Fino ad ora abbiamo costruito applicazioni **monolitiche**:

- **un unico progetto con Entity, DTO, DAO, Service, Controller e Mapper tutti insieme.**

Questo approccio funziona bene per applicazioni piccole, ma nei sistemi reali a livello commerciale non è conveniente.

Immagina un e-commerce: avresti un unico progetto gigantesco con il catalogo prodotti, i pagamenti, gli ordini, la gestione utenti, le spedizioni — tutto insieme. Il problema non è tecnico ma **commerciale e organizzativo**:

- **team diversi devono lavorare su parti diverse dell'applicazione, con ritmi e cicli di deploy diversi.**

Se hai poche richieste al giorno, un'architettura monolitica può avere senso. Ma per sistemi come Amazon o Netflix — che gestiscono milioni di richieste ogni giorno — i microservizi sono la scelta obbligata.

Cos'è un'Applicazione a Microservizi

Un'applicazione a microservizi è un'applicazione composta da tanti **piccoli servizi indipendenti**, ognuno con:

1. Una singola responsabilità:

- Ogni micro-servizio fa una sola cosa — gestisce il catalogo, oppure i pagamenti, oppure gli ordini.
- La suddivisione avviene in base al dominio di business e al carico di lavoro.

2. Proprio ciclo di vita e deploy:

- Ogni micro-servizio può essere aggiornato, spento e riavviato indipendentemente dagli altri.

≡ **Ad esempio una banca che deve aggiornare il servizio di prelievo può spegnere solo quella parte — senza mandare offline l'intera applicazione.**

- In un'architettura monolitica questo non sarebbe possibile.

3. Propria base dati:

- Ogni micro-servizio ha il proprio database — o almeno la porzione di database che gli serve.

≡ **Ad esempio il servizio degli ordini, una volta inserito un ordine, deve comunicare con il servizio dei pagamenti — che ha il suo database separato.**

4. Proprio meccanismo di comunicazione:

I microservizi comunicano tra loro in due modi:

- **API REST** sincrone:
 - il chiamante aspetta la risposta prima di procedere
- Messaging asincrono:
 - il chiamante invia un messaggio e non aspetta la risposta

≡ **Un esempio concreto di messaging asincrono:**

Quando prenoti un biglietto per uno spettacolo e paghi, ricevi un'email di conferma.

In realtà il servizio di prenotazione, il servizio di pagamento e il servizio di invio email sono tre microservizi separati.

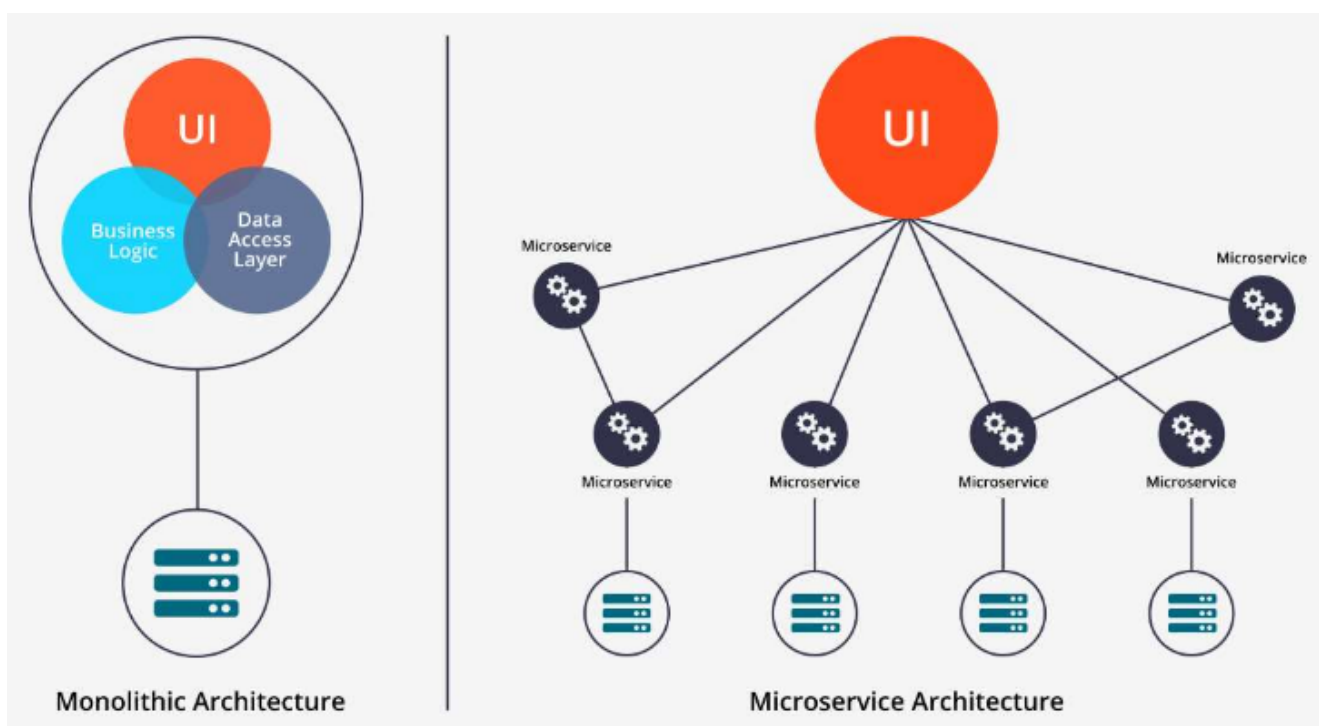
Il servizio email non sa perché o a chi deve inviare — riceve solo il messaggio e lo invia.

La transazione non è considerata completa finché tutti e tre i servizi non hanno terminato il loro lavoro.

👁️ **Nota:**

Non tutti i microservizi hanno un'interfaccia grafica — alcuni sono servizi interni che vengono interpellati solo da altri microservizi.

Ad esempio quando acquistiamo un prodotto online, il servizio che gestisce la spedizione e assegna il corriere è un micro-servizio interno — noi non lo vediamo mai direttamente.



Monolite vs Microservizi — Il Confronto Visivo

L'immagine mette a confronto le due architetture in modo molto chiaro.

1. A sinistra — Architettura Monolitica:

- UI, Business Logic e Data Access Layer sono tre cerchi sovrapposti — **un unico blocco indivisibile che condivide tutto:**
 - **lo stesso codice, lo stesso deploy e lo stesso database.**
- **Se si vuole aggiornare anche solo la Business Logic, bisogna toccare e ridistribuire l'intero sistema.**

2. A destra — Architettura a Microservizi:

- **La UI è un unico punto di ingresso che si collega a tanti microservizi indipendenti, ognuno con il proprio database.**
- **Si nota subito che i microservizi sono organizzati in modo gerarchico — alcuni microservizi chiamano a loro volta altri microservizi.**

☰ **Ad esempio:**

la UI potrebbe chiamare il microservizio degli ordini, che a sua volta chiama il microservizio dei pagamenti, che a sua volta chiama il microservizio delle notifiche email.

Ogni microservizio ha:

- Il proprio **ingranaggio** — **la propria logica e il proprio ciclo di vita**
- Il proprio **database** — **indipendente dagli altri**

📄 **Il vantaggio è evidente:**

✓ **se si vuole aggiornare il microservizio dei pagamenti, si tocca solo quello — la UI e tutti gli altri microservizi continuano a funzionare indisturbati.**

⚠ **Nel monolite invece anche un piccolo aggiornamento richiede di ridistribuire l'intera applicazione.**

Applicazione Cloud

Un'applicazione cloud è un software progettato per:

- **Funzionare su infrastrutture remote** accessibili via internet:

- **non gira su un singolo server fisico ma su macchine virtuali distribuite geograficamente**
- **Adattarsi automaticamente al carico di lavoro:**
 - **se le richieste aumentano, l'infrastruttura si scala automaticamente aggiungendo nuove istanze del servizio; se diminuiscono, le istanze vengono ridotte per ottimizzare i costi**
- **Garantire alti livelli di sicurezza e facilità di aggiornamento:**
 - **gli aggiornamenti vengono distribuiti in modo graduale senza interruzioni del servizio**

Il Problema della Scalabilità

La scalabilità è un problema cruciale — ma non per il programmatore, bensì per **l'azienda che deve fatturare**.

Gestire picchi di traffico improvvisi (come il Black Friday per un e-commerce) senza sovradimensionare l'infrastruttura nei periodi di basso traffico è una sfida puramente economica e organizzativa.

Le operazioni di **scaling**; **ovvero aggiungere o rimuovere istanze dei microservizi in base al carico** — vengono gestite automaticamente da Kubernetes:

- **lo strumento standard per orchestrare i container in ambiente cloud.**

Questo è il motivo per cui i microservizi si sposano perfettamente con il cloud:

- **ogni micro-servizio può essere scalato indipendentemente dagli altri.**

Se il servizio dei pagamenti riceve più traffico del solito, si scala solo quello — senza toccare il servizio del catalogo o degli ordini.

Caratteristiche del Cloud in Dettaglio

Ora che abbiamo capito cosa sia un'applicazione cloud e perché i microservizi si sposano naturalmente con essa, vale la pena analizzare nel dettaglio le caratteristiche che la rendono così vantaggiosa rispetto a un'architettura tradizionale on-premise.

Il punto di partenza è semplice:

- un'azienda che sviluppa software non vuole occuparsi di gestire server fisici, aggiornamenti manuali, picchi di traffico improvvisi o infrastrutture di sicurezza

complesse.

Vuole concentrarsi sul **prodotto** — e delegare tutta l'infrastruttura a chi lo fa di mestiere.

È esattamente questo il valore del cloud.

1. Esegue servizi e calcoli su server remoti:

- Il carico di lavoro non pesa sui server locali aziendali — detti **on-premise** — ma sul cloud.
- Questo significa che l'azienda non deve investire in hardware proprio:
 - il cloud mette a disposizione potenza di calcolo, spazio di archiviazione e servizi aggiuntivi come database gestiti, analytics e AI.

✓ **Si paga solo ciò che si usa, quando si usa.**

2. Aggiornamenti automatici e continui:

- L'applicazione viene aggiornata automaticamente sul server cloud — lo sviluppatore aziendale non deve installare nulla, non deve distribuire manualmente nessun aggiornamento.
- Questo si collega direttamente al concetto di **ciclo di vita indipendente** dei microservizi:
 - ogni servizio può essere aggiornato autonomamente senza interruzioni per gli altri.

3. Scalabilità:

- È forse la caratteristica più importante dal punto di vista commerciale.
- Se il numero di utenti aumenta improvvisamente — come durante un Black Friday — il cloud risponde automaticamente in tre modi:
 1. **Aumentando le risorse:**
 - disponibili per i servizi sotto pressione
 2. **Replicando i servizi:**
 - creando nuove istanze dello stesso microservizio per distribuire il carico
 3. **Bilanciando il carico:**
 - distribuendo le richieste in arrivo tra le varie istanze in modo uniforme

È Kubernetes che orchestra tutte queste operazioni automaticamente, senza intervento umano.

4. Sicurezza Integrata:

- Il cloud fornisce un'infrastruttura di sicurezza completa che sarebbe costosa e complessa da implementare on-premise:
 - **Cifratura dei dati** — i dati sono protetti sia in transito che a riposo
 - **Gestione delle identità** — controllo degli accessi e autenticazione centralizzata
 - **Backup automatici** — i dati vengono salvati periodicamente senza intervento manuale
 - **Firewall avanzati** — protezione contro attacchi esterni

Tipi di Applicazioni Cloud

Le applicazioni cloud non sono tutte uguali:

- si distinguono in base a quanto dell'infrastruttura viene gestito dal provider e quanto rimane in carico all'azienda.

Esistono tre modelli principali, in ordine decrescente di astrazione:

1. **SaaS — Software as a Service:**

- È il livello più alto di astrazione:
 - l'applicazione è già completamente implementata e pronta all'uso — si accede tramite browser o app mobile senza installare nulla.
 - L'utente finale non sa e non gli importa dove gira il software, su quanti server, con quale tecnologia.

Esempi: Google Drive, Microsoft Teams, Salesforce.

2. **PaaS — Platform as a Service:**

- Il provider mette a disposizione una piattaforma su cui gli sviluppatori possono costruire e distribuire le proprie applicazioni, senza preoccuparsi di gestire l'infrastruttura sottostante — server, sistema operativo, runtime.
- Lo sviluppatore si concentra solo sul codice.

Esempi: Google App Engine, Heroku.

3. **IaaS — Infrastructure as a Service:**

- È il livello più basso:
 - il provider offre le risorse hardware virtualizzate — macchine virtuali, reti, storage.
 - L'azienda ha il massimo controllo ma anche la massima responsabilità:
 - deve gestire autonomamente sistema operativo, runtime e applicazione.

Esempi: **AWS EC2, Azure VM.**

👁 **Nota la progressione:**

dal **SaaS** all'**IaaS** aumenta il controllo ma anche la complessità di gestione. Per un'azienda che vuole concentrarsi sul prodotto senza occuparsi di infrastruttura, **SaaS** e **PaaS** sono le scelte più convenienti. Per chi ha esigenze specifiche di configurazione e controllo totale, **IaaS** è la scelta giusta.

Applicazione Cloud Multi Tenant

Tra le caratteristiche più interessanti delle applicazioni cloud c'è il concetto di **multi tenancy**.

Fino ad ora abbiamo parlato di cloud come infrastruttura scalabile e distribuita — ma non abbiamo ancora risposto a una domanda fondamentale: se migliaia di aziende usano lo stesso servizio cloud, come fa il provider a gestire tutti questi clienti in modo efficiente senza moltiplicare i costi infrastrutturali? La risposta è proprio il multi tenancy.

Cos'è un Tenant?

Un **tenant** è:

- un cliente o un'organizzazione che utilizza un servizio cloud.
- Il termine viene dall'inglese — come un inquilino che affitta un appartamento in un condominio.

Cos'è il Multi Tenancy?

Un'applicazione è **multi tenant** quando più clienti — detti appunto tenant:

- **condividono la stessa infrastruttura e la stessa applicazione, mantenendo però i propri dati e configurazioni completamente isolati dagli altri.**

In pratica: Azienda A, Azienda B e Azienda C usano lo stesso software sugli stessi server — ma ognuna vede solo i propri dati e non può accedere a quelli delle altre.

☰ **È esattamente come un condominio: gli inquilini condividono lo stesso edificio, le scale e l'ascensore — ma ognuno ha il proprio appartamento privato a cui gli altri non possono accedere.**

Dove si Applica

Le applicazioni **SaaS** e **PaaS** sono tipicamente multi tenant. Un esempio concreto è **GitHub**:

- migliaia di sviluppatori e aziende usano la stessa infrastruttura, ma ognuno vede solo i propri repository — e quelli privati sono completamente isolati dagli altri utenti.

✓ **Il vantaggio per il provider è evidente:**

- **invece di mantenere un'infrastruttura separata per ogni cliente, ne mantiene una sola condivisa — riducendo enormemente i costi operativi.**

✓ **Il vantaggio per il cliente è altrettanto chiaro:**

- **paga solo per ciò che usa, senza dover gestire nulla dell'infrastruttura sottostante.**

Caratteristiche delle Applicazioni Cloud a Microservizi

Mettendo insieme tutto ciò che abbiamo visto — i microservizi, il cloud e il multi tenancy — arriviamo al paradigma architetturale più efficace per i sistemi moderni: **le applicazioni cloud a microservizi.**

La progettazione di applicazioni cloud basate su microservizi è oggi uno degli approcci più diffusi per creare sistemi:

- **Scalabili** — ogni microservizio può essere scalato indipendentemente in base al carico
- **Resilienti** — se un microservizio cade, gli altri continuano a funzionare
- **Facilmente evolvibili** — ogni microservizio può essere aggiornato, sostituito o riscritto senza impattare il resto del sistema

Questo approccio sfrutta al massimo le potenzialità del cloud:

- invece di deployare un'unica applicazione monolitica, si suddivide il sistema complesso in una serie di **servizi autonomi, indipendenti e specializzati**:
 - ognuno con la propria responsabilità, il proprio database e il proprio ciclo di vita.

Il risultato è un'applicazione composta da microservizi **interoperabili** — che comunicano tra loro tramite [API REST](#) o [messaging asincrono](#) — e con un **alto livello di indipendenza** reciproca.

Non è un caso che Amazon, Netflix e Google abbiano adottato questo paradigma:

- è l'unico approccio che permette di gestire sistemi di quella complessità e scala mantenendo team autonomi e cicli di rilascio rapidi.

Scalabilità di un applicazione a microservizi

Supponiamo di avere un'applicazione a microservizi caricata in cloud su un **cluster** — un insieme di nodi (computer, server, ecc.) che cooperano in rete come un unico sistema logico.

La **scalabilità**:

- è la capacità del sistema di reagire all'aumento del carico di lavoro.
Non esiste un unico modo per scalare — esistono tre approcci principali, ognuno con caratteristiche diverse.
 1. **Scalabilità Orizzontale**:
- Si aumenta il numero di istanze dello stesso servizio all'interno dello stesso cluster — senza aggiungere nuovi cluster.

☰ **Analogia McDonald's:**

aumentano i clienti in un ristorante — si aprono più casse nello stesso locale.

È l'approccio più immediato:

- **si replica il microservizio più sotto pressione e si distribuisce il carico tra le istanze.**
- Kubernetes gestisce questa operazione automaticamente.
- 2. **Scalabilità Orizzontale + Mirroring:**
- **Non solo si aumentano le istanze, ma si replica l'intera applicazione su più cluster distribuiti geograficamente.**

☰ **Analogia McDonald's:**

aumentano i clienti ovunque — si aprono nuovi ristoranti in più città, e in ogni ristorante si aggiungono casse in base al carico locale.

È l'approccio usato da sistemi come Amazon e Netflix:

- l'applicazione gira su cluster in Europa, America, Asia — così le richieste vengono servite dal cluster geograficamente più vicino all'utente, riducendo la latenza.

3. **Scalabilità Verticale:**

- **Invece di aggiungere nuove istanze o nuovi cluster, si aumenta la potenza di un singolo nodo — più CPU, più RAM, più storage.**

☰ **Analogia McDonald's:** aumentano i clienti — si chiede allo staff di lavorare più velocemente e ottimizzare i tempi.

È l'approccio più semplice da implementare ma ha un limite fisico evidente:

- **una macchina non può crescere all'infinito.**
- **Per questo nei sistemi moderni la scalabilità verticale viene usata come complemento alle altre, non come soluzione principale.**

Premessa al Concetto di Resilienza

Lo Scenario

In un'applicazione a microservizi, ogni microservizio non lavora in isolamento — **per realizzare le proprie funzionalità di business deve interfacciarsi con altri microservizi.**

Guardando l'immagine, il flusso è il seguente:



1. Il **Client** invia una richiesta al **Gateway**:

- **il punto di ingresso unico dell'applicazione, che fa da "portiere" smistando le richieste ai microservizi corretti**

2. Il **Gateway**:

- inoltra la richiesta al **Microservizio** principale

3. Il Microservizio deve a sua volta:

- **comunicare con altri microservizi per completare la sua operazione — e lo può fare in due modi:**

1. **Sincrona (HTTP):**

- comunicazione diretta verso il **Microservizio A**: **il microservizio chiama e aspetta la risposta prima di procedere.**

1. **Asincrona (Coda messaggi):**

- comunicazione tramite una **coda di messaggi** verso il **Microservizio B**: **il microservizio invia il messaggio nella coda e non aspetta la risposta — il Microservizio B la processerà quando potrà.**

Il Problema

Questo scenario introduce un problema fondamentale: **cosa succede se il Microservizio A non risponde?**

Nella **comunicazione sincrona** il chiamante aspetta — **se il Microservizio A è down o lento, l'intera catena si blocca.**

In un sistema monolitico questo problema non esiste — tutto è nello stesso processo.

In un sistema a microservizi invece un singolo punto di fallimento può propagarsi a cascata e bloccare l'intera applicazione.

È esattamente per rispondere a questo problema che nasce il concetto di **resilienza**:

- **la capacità del sistema di continuare a funzionare anche in presenza di guasti parziali.**

Lo vedremo nella sezione successiva.

Resilienza di un'Applicazione a Microservizi

La resilienza è la risposta al problema che abbiamo appena posto: cosa succede se un microservizio non risponde? Un sistema resiliente non crolla — si adatta. Si traduce in tre sotto-caratteristiche:

1. Fault Tolerance — Sopravvivenza al Guasto:

- **Un microservizio deve contemplare la possibilità che l'interazione con un altro microservizio possa fallire e avere già a disposizione un piano B — senza cambiare la natura del risultato.**

☰ Esempio:

il microservizio dei suggerimenti di Amazon chiama il microservizio della cronologia acquisti per personalizzare i consigli.

Se quel microservizio non risponde, invece di restituire un errore al client, il piano B potrebbe essere restituire i prodotti più venduti in generale — il risultato non è personalizzato ma è comunque un risultato valido.

2. Bulkhead — Isolamento:

- **Poiché l'applicazione è suddivisa in microservizi indipendenti, il fallimento di uno non comporta necessariamente l'indisponibilità dell'intera applicazione.**

☰ Analogia:

è come le paratie stagne di una nave — se un compartimento si allaga, le paratie impediscono all'acqua di invadere il resto della nave.

Allo stesso modo, se il microservizio dei pagamenti va down, il catalogo prodotti e la ricerca continuano a funzionare.

✓ **Questo è uno dei vantaggi fondamentali rispetto all'architettura monolitica:**

nel monolite un guasto può bloccare tutto, nei microservizi il danno è circoscritto.

3. Graceful Degradation — Degradazione Aggraziata:

- Di fronte al fallimento di una chiamata a un microservizio, il sistema utilizza un piano B che restituisce un risultato simile ma non necessariamente equivalente a quello richiesto.

✓ **La differenza rispetto alla Fault Tolerance è sottile ma importante:**

la Fault Tolerance mantiene il risultato invariato tramite il piano B, la Graceful Degradation accetta che il risultato sia degradato — meno preciso, meno personalizzato, meno completo — ma pur sempre utile per l'utente.

☰ **Esempio:**

Netflix non riesce a caricare i suggerimenti personalizzati — invece di mostrare una pagina di errore, mostra i contenuti più popolari del momento. L'esperienza è degradata ma l'utente può comunque usare il servizio.

☰ **In sintesi:**

un sistema resiliente non è un sistema che non fallisce mai — **è un sistema che sa come comportarsi quando fallisce.**

Fault Tolerance, Bulkhead e Graceful Degradation sono i tre strumenti con cui i microservizi garantiscono che un guasto parziale non diventi mai un guasto totale.

Facilità di Evoluzione di un'Applicazione a Microservizi

La terza caratteristica fondamentale delle applicazioni cloud a microservizi — insieme a scalabilità e resilienza — è la **facilità di evoluzione**.

È forse la caratteristica più importante dal punto di vista commerciale:

- **un sistema che evolve facilmente è un sistema che risponde rapidamente alle esigenze del mercato.**

Un'applicazione a microservizi è facilmente evolvibile perché ogni servizio:

- È **autonomo**:
 - **ha la propria logica, il proprio ciclo di vita e il proprio team di sviluppo**
- Ha un **proprio database**:
 - **non condivide dati con gli altri microservizi, quindi le modifiche alla struttura dati restano localizzate**
- È **deployato in modo indipendente**:
 - **si può aggiornare, sostituire o riscrivere un microservizio senza toccare gli altri**
- È **scalabile in modo selettivo**:
 - **si scala solo il microservizio che ne ha bisogno, non l'intera applicazione**
- Ha un **coupling debole** con gli altri microservizi:
 - **comunica tramite API o messaggi, non tramite dipendenze dirette nel codice**

Conclusione:

le modifiche restano **localizzate** e non impattano l'intero sistema. Questo è l'esatto opposto dell'architettura monolitica — dove una modifica anche piccola richiede di testare e ridistribuire l'intera applicazione. Nei microservizi si modifica, si testa e si deploya **un servizio alla volta**, in modo rapido e sicuro